

Lab 7: Container ADTs

Due **Sunday, October 29th, 2023 at 8:00 PM ET** on Gradescope.

Each lab is graded for attendance (individual) and completion (group). **Note: Don't forget to add your pod members to the Gradescope submission!**

Getting started

1. One of your group members should create an editable copy of this worksheet to fill out and share the document with all other group members. (*File → Make a Copy* in the menu.)
2. Fill in the table below based on the letters given to each group member. If your group does not have exactly four members, please work out a strategy for sharing the roles that works for you.

	Name	Role (rotates each week)
A	Sonya Konon	Leader
B	Shikha Nukala	Scribe
C	Spence Lowery	Coder I
D		Coder II

3. The **scribe** should do a find/replace (select *Edit > Find and replace* from the menu) to change all instances of "Person A" to that member's name, "Person B" to that member's name, and so on.

Formatting

Please do not change the formatting of the document or which exercises appear on which slides. We use the placement of questions on each page to assist with grading.

Questions?

You are expected to lead the meeting yourselves and collaboratively work through the exercises, but please let us know if you have questions! We're here if you need us and will be checking in throughout the lab.

Note
These lab exercises were developed for in-person and virtual labs. Depending on the way you and your group are working, you are welcome to interpret the exercise instructions however works best (e.g. "share screen" means something different if you're in-person vs. virtual).

Implementing a Ring Buffer

In this exercise, you'll implement the core member functions for a RingBuffer class.

Getting Started

Open a terminal and navigate to the folder where you keep your EECS 280 stuff. Then, use git to clone the starter files from our repository:

```
git clone https://github.com/jamesjuett/eeecs280_ringbuffer_lab.git
```

That will create a folder called eeecs280_ringbuffer_lab containing all the files for this exercise. You don't need to do anything else with git or GitHub for this lab.

If you want to work from the terminal, you'll need to cd into the project folder. Or, if you're using VS Code, you can open the folder with:

```
code eeecs280_ringbuffer_lab
```

As with previous labs, we encourage your group to collaborate with VS Code live share. More info and instructions are at:

<https://code.visualstudio.com/learn/collaboration/live-share>

Implement RingBuffer Functions

Take a look at the general structure of the RingBuffer class in RingBuffer.hpp. Note the member variables in the private section (head, tail, num_elts, capacity), which follow from the data representation introduced at the beginning of lab (you may refer back to the slides as you work). Many functions are written for you.

Implement the remaining functions in RingBuffer.hpp. You can check your implementation with the tests we've provided:

```
make RingBuffer_tests.exe
./RingBuffer_tests.exe
```

When you're finished, paste your implementation in the boxes here.

```
void push_back(const T &value) {
```

```
void push_back(const T &value) {
    if(num_elts == (capacity - 1)){
        grow();
    }
    data[tail] = value;
    tail = (tail+1) % capacity;
    num_elts++;
}
```

```
}
```

```
void pop_front() {
```

```
void pop_front() {
    data[head] = 0;
    head = (head+1) % capacity;
    num_elts--; // TODO: Replace with your
    implementation
}
```

```
}
```

```
T & front() {
```

```
T &front() {
    return data[head];
}
```

```
}
```

RingBuffer vs. std::vector Performance Showdown

Now, let's compare the performance of your RingBuffer against std::vector for implementing a queue. Take a quick look at the file performance.cpp.

It takes in command line arguments to select a type of container (either "vector" or "ringbuffer") and an input size N. Then, it initializes the chosen container with N elements and performs N iterations of push/pop operations on each.

Compile the program with:

```
make performance.exe
```

Then, use the time command to measure the execution time for various input sizes. For example, to measure the time taken for a std::vector with N = 100000:

```
time ./performance.exe vector 100000
```

Or, to measure the time taken for a RingBuffer with N = 400000:

```
time ./performance.exe ringbuffer 400000
```

Using this approach, fill in the tables below. Use the "user" time reported.

std::vector				
N	100000	200000	300000	400000
time (seconds)	.74	2.84	6.39	11.5

RingBuffer					
N	100000	200000	300000	400000	10000000 (10 million)
time (seconds)	0.00	.01	.01	.01	.16

Explain the times you found in terms of the time complexity for these operations based on N (which is the size of the queue as well as the number of operations performed).

For vector, the time complexity is probably something like $O(N)$ or worse because it is increasing by more than double when N is doubled. Ringbuffer is probably something like $O(\log n)$ or something even better like $O(1)$ because it is extremely fast and basically does not increase from one hundred thousand to ten million.

Operator Overloading

Let's say we would like to implement an overloaded `[]` operator that allows access to the element at a particular position in a `RingBuffer`. For example:

```
RingBuffer<int> queue;  
queue.push_back(1);  
queue.push_back(2);  
queue.push_back(7);  
  
cout << queue[0] << endl; // prints 1  
cout << queue[2] << endl; // prints 7  
queue[0] = 10; // first element in the queue is now 10
```

Fill in the boxes below to implement the `[]` operator as it would appear inside the `RingBuffer` class. Note the two versions - a regular and a const-qualified version. (Hint: you can use the `.at()` function in your implementation.)

```
T operator[] ( Int index ) {
```

```
    Return data.at(index);
```

```
}
```

```
T operator[] ( Int index ) const {
```

```
    Return data.at(index);
```

```
}
```

In the parameter lists for the `operator[]` functions above, we don't pass in a parameter of type `const RingBuffer &`. However, in the overload implementation for the `<<` operator (refer to the code in `RingBuffer.hpp`), such a parameter is included. Explain the reason for this difference in the box below.

When you call the `[]` operator, you would call it like `RingBuffer rb; rb[0]`. It is kind of like just calling `rb.at()` which you also don't need a `RingBuffer` parameter because you are calling the specific object's method. So when you call a specific objects method you dont need it in the parameter. For something like `<<` you do need

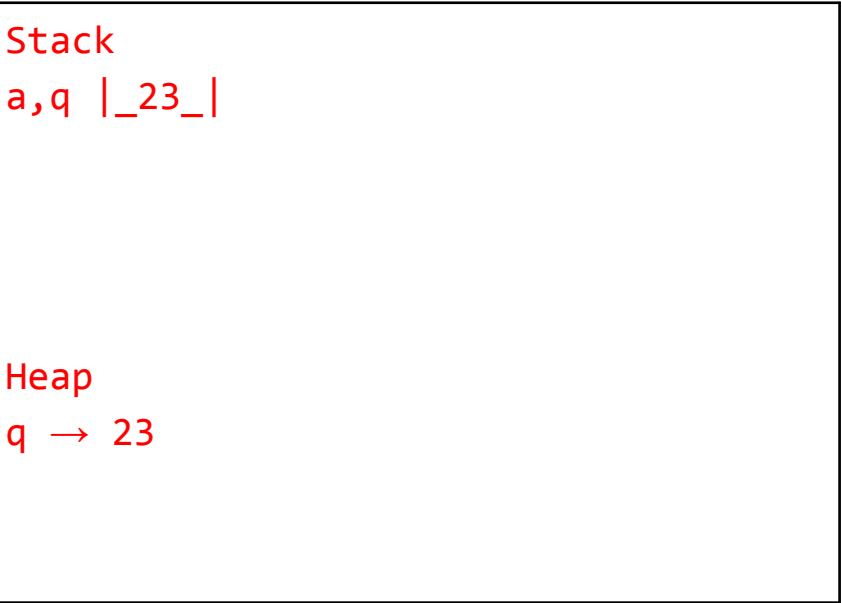
Dynamic Memory

The **scribe** should draw a memory diagram of the stack and heap right before func returns, with help from all group members.

You might want to use some separate drawing program and paste a screenshot of your image in the box below.

```
void func() {
    int a = 42;
    int *p = &a;
    int *q = new int(65);
    p = q;
    q = new int(23);
    delete p;
    q = &a;

    // DRAW MEMORY HERE
}
```



The code above contains a memory leak. What is the value of the leaked object?	We have to delete q because it was initialized as a new variable.
Could you add a line to the end of func() to ensure all memory is freed? If so, what line? If not, why not?	We were hoping to do delete all, but we can't because then we would be double deleting variables, which would throw us an error, therefore no, there is not a line we could add.

Dynamic Memory Errors!

Several main functions are given below. **All of them compile successfully**, but **all also contain** an error relating to dynamic memory, for example:

- memory leak
- incorrect delete
- double delete
- use of uninitialized value
- dereferencing a null pointer
- use of dangling pointer or reference to a dead object

Discussion Leader	Code Snippet	What dynamic memory error occurs? Justify your answer.
Person D	<pre>int *a = new int(7); int &b = *a; delete a; cout << b << endl;</pre>	Cannot print b after deleting a because b is a reference to a -> dangling pointer
Person A	<pre>int *a; int *b = new int(1); a = b; b = a; cout << *a << endl; cout << *b << endl; delete a; delete b;</pre>	Bad delete, cannot delete a because its not a new variable
Person B	<pre>int *a = new int(6); cout << *a << endl; delete a; a = new int[7]; *a = 3; cout << *a << endl; delete a;</pre>	Double delete, would need to reinitailize int a
Person C	<pre>int *a = new int(54); *a = *new int(10); cout << *a << endl; delete a;</pre>	Memory leak

The Big Three - Warm Up Questions

Question	Discussion Leader	Answer
Which special member functions comprise the Big Three?	Person D	Destructor Copy Constructor Assignment Operator
Give an example of a class that would need a custom destructor implementation. Why is it needed?	Person A	<pre>class RingBuffer { public: RingBuffer() : data(new int[INITIAL_CAPACITY]) { } ~RingBuffer() { delete[] data; } }</pre> Needed because we initialize variable as new to the heap
For the class above, would a shallow copy or a deep copy be appropriate? Why?	Person B	Shallow copy because you can delete the actual variable initialized rather than just a copy.
What is the "rule of the Big Three", and how does it relate to your answers above?	Person C	The rule of three is that if you need any one of these, you almost certainly need all of them.

RingBuffer Dynamic Memory and Big Three

Let's return to the RingBuffer class you implemented earlier. It turns out it doesn't properly manage its dynamic memory or follow the rule of the Big Three.

Memory Leaks

First, check for memory leaks while running RingBuffer_tests.exe. If you're on Windows or Linux, use the address sanitizer. If you're on a Mac, use the leaks tool. Instructions for both are in our [Leak Checking Tutorial](#) (available on eecs280.org > Tutorials). You'll want to use these to check your project code for leaks as well!

Heads up! When you modify the Makefile to enable sanitizer tools, you'll likely need to run make clean and then make RingBuffer_tests.exe. (Otherwise, make may not recognize it needs to recompile, since the source code itself is unchanged.)

You'll find that there are memory leaks! Part of the output should include a summary of the leaks. the total number of bytes leaked. For example:

(Windows/Linux) SUMMARY: AddressSanitizer: X byte(s) leaked in Y allocation(s).
OR
(Mac) Y leaks for X total leaked bytes.

How many leaks are there?	9
---------------------------	---

How many bytes are leaked total?	176
----------------------------------	-----

To fix the memory leak, add a custom destructor implementation. Recompile your code and verify there are no more leaks. Paste your destructor implementation in the box:

```
~RingBuffer() {  
    delete [] data;  
}
```


RingBuffer Dynamic Memory and Big Three (Continued)

The RingBuffer class is also missing a custom **assignment operator**. For its implementation, we have provided a helper function that does some of the work:

```
// REQUIRES: this RingBuffer has capacity >= other.num_elts
// EFFECTS: Copies all elements from the other ring buffer into this one,
//           replacing any existing elements. They are copied to beginning
//           of the data array and the head/tail are adjusted accordingly.
void copy_data_from(const RingBuffer &other) {

    // Copy elements from other, placing at the start of our own array
    for(int i = 0; i < other.num_elts; ++i) {
        data[i] = other.at(i);
    }

    // Adjust head/tail and num_elts to match new data
    head = 0;
    tail = other.num_elts;
    num_elts = other.num_elts;
}
```

Consider the potential implementations below. In the boxes, either write "correct" or a brief description of what is wrong with the code.

If you'd like to check them, find and uncomment the corresponding test in RingBuffer_tests.cpp. Then you can paste each into your RingBuffer.hpp file.

```
RingBuffer &operator=(
    const RingBuffer &rhs) {

    if (this == &rhs) { return *this; }
    capacity = rhs.capacity;
    data = new T[rhs.capacity];
    copy_data_from(rhs);
    delete[] data;
    return *this;
}
```

Correct

```
RingBuffer &operator=(
    const RingBuffer &rhs) {

    if (this == &rhs) { return *this; }
    capacity = rhs.capacity;
    delete[] data;
    data = new T[rhs.capacity];
    copy_data_from(rhs);
    return *this;
}
```

Wrong because you are deleting data before data even exists yet.

```
RingBuffer &operator=(
    const RingBuffer &rhs) {

    if (this == &rhs) { return *this; }
    capacity = rhs.capacity;
    delete[] data;
    data = rhs.data;
    copy_data_from(rhs);
    return *this;
}
```

Deleted data before copying the rhs, will throw an error because the variable does

```
RingBuffer &operator=(
    const RingBuffer &rhs) {

    if (this == &rhs) { return *this; }
    capacity = rhs.capacity;
    delete data;
    data = new T[rhs.capacity];
    copy_data_from(rhs);
    return *this;
}
```

Delete data before setting it to a new variable. cannot delete what does not

Submit worksheet

Your pod is responsible for submitting lab worksheets to Gradescope by the deadline. We strongly recommend submitting your worksheet on your lab day so you don't forget!

Download a PDF copy of your completed worksheet by going to *File > Download > PDF*.

The ***Scribe*** is responsible for submitting the PDF to Gradescope. **You need to add your group members to the submission after it is uploaded:**

